Using Frontier Models Without Giving Up Your Data

rizzo-pii:0.3B: a lightweight, CPU-friendly, Italian-first model for local and reversible PII anonymization

Simone Rizzo

Rizzo AI Academy · rizzoaiacademy.com

Code & data: github.com/Rizzo-AI-Academy/rizzo-pii

100% LOCAL · GDPR BY DESIGN · EU AI ACT ALIGNED

Technical report · June 2026

Abstract

Millions of people now paste e-mails, contracts, court filings and other sensitive documents into closed frontier assistants such as ChatGPT, Claude and Gemini. That text leaves the user's machine and travels to third-party servers, where it may be logged, cached, processed or retained, a poor fit for legal practice, healthcare or anyone bound by the GDPR. Running a comparable model locally is the obvious cure, but a frontier-grade open model needs a workstation that most individuals and small firms cannot afford, while the small models that **do** fit a laptop are not competitive on demanding Italian legal text. We propose a different trade-off: keep using the best closed models, but never send them raw data. **rizzo-pii:0.3B** is a token-classification model (≈0.3B parameters, ModernBERT/mmBERT backbone) that runs on a **CPU** in roughly **0.5 GB of RAM**, detects 22 categories of personal data (including Italian-specific identifiers such as **codice fiscale**, **partita IVA** and **dati catastali** that no other open model covers) and drives a fully **reversible** anonymization workflow: redact locally, query the frontier model with placeholders, then reconstruct the real values from a local dictionary. On a held-out, real-text Italian benchmark the model reaches **0.989 micro-F1** (0.987 precision, 0.990 recall). We describe the model, the 745k-row multilingual training corpus, the "LLM-author / code-labeler" synthetic-data method, the results, and an open call to build a large, community-owned Italian PII dataset.

1 The problem: convenience is leaking your data

Large closed assistants have become everyday tools. People summarize contracts, draft replies to e-mails, translate medical reports and ask legal questions by simply **pasting the document in**. It is fast, it is useful, and it quietly moves enormous amounts of personal and confidential data off the user's device. Names, addresses, tax codes, IBANs, health details, clauses of unsigned contracts: all of it crosses the network to servers the user does not control, where it may be retained, used for analytics, or exposed in a breach. For a law firm, an accountant or a hospital, this is not a hypothetical risk; under the GDPR it can be a direct compliance failure, because the data subject's information is transferred to third parties without a sound legal basis or adequate safeguards.

The intuitive fix is to **stop sending data out** and run an open model locally. The difficulty is cost. A frontier-grade open model is large: serving a state-of-the-art mixture-of-experts model (a DeepSeek-class model, for instance, in setups reportedly built on 128 GB of unified memory such as Salvatore Sanfilippo's

DwarfStar) lands around **€9,000–€10,000** of hardware, out of reach for most professionals and individuals. The models that comfortably fit a normal machine (the small open models in the 2B–12B class, e.g. the Gemma family) are genuinely useful, but they are **not** in the same league as ChatGPT, Claude or Gemini precisely on the hard tasks: legal reasoning, statutes, dense contracts and long official documents, which is exactly where Italian professionals need the most help.

The trade-off we actually want

We do not have to choose between **capability** (frontier closed models) and **privacy** (small local models). We can keep the frontier model and remove the data from the equation: anonymize the document **locally** on a CPU, send only placeholders to the cloud, and restore the real values **locally** from the answer. The sensitive content never leaves the machine.

2 rizzo-pii in one picture

The workflow has three local steps and one remote step. Locally, rizzo-pii tags every span of personal data and replaces each one with a stable, type-aware placeholder (`[FULLNAME_1]`, `[IBAN_1]`, `[CF_1]`), recording the mapping `placeholder → real value` in a dictionary that **stays on disk**. Identical values share the same placeholder, so the frontier model still sees a coherent text and can reason about it. The anonymized text (and only the anonymized text) is sent to ChatGPT / Claude / Gemini. When the answer comes back, a local pass swaps the placeholders for the true values using the dictionary. The cloud provider never receives a single real name, code or number.

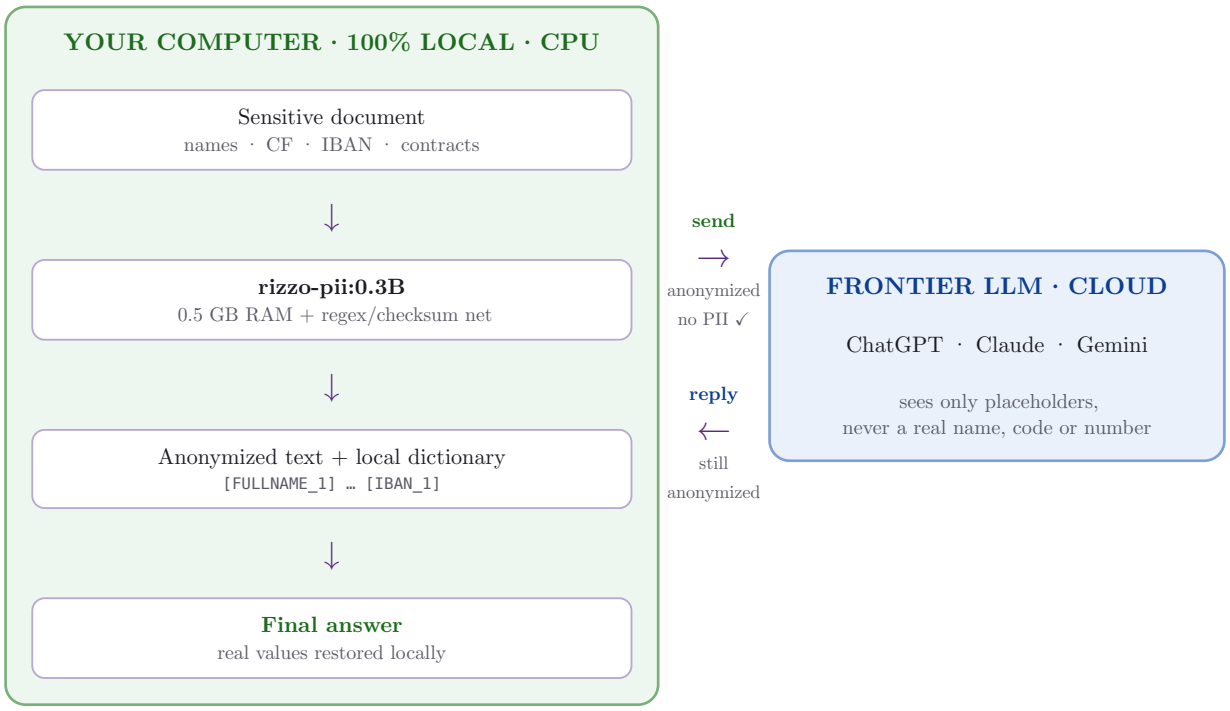


Figure 1: The rizzo-pii workflow. Everything except the frontier query happens on the user's CPU; only placeholder text crosses the boundary, and the answer is re-identified locally.

3 Why this is different: privacy that is actually private

The point of rizzo-pii is not "yet another PII detector". It is an **architecture for using powerful models without surrendering data**, and it is built so that the privacy guarantee is structural rather than a promise:

- **The data never leaves the device.** Detection and re-identification run locally on a CPU. There is no API key, no telemetry, no upload. What the cloud receives is already stripped of identifiers.

- **GDPR by design.** The workflow implements **data minimization** (Art. 5) almost literally: the third-party processor only ever sees pseudonymized text, so the most common reason a cloud LLM call is unlawful (transferring identifiable personal data to a third party / third country without basis) is removed at the source.
- **Aligned with the EU AI Act.** Keeping personal data under local control and out of third-party model pipelines supports the Act's emphasis on data governance and on protecting individuals' rights when AI systems are used.
- **Accessible to everyone.** Because the model is 0.3B parameters and runs on a CPU in well under 1 GB of RAM, the privacy layer costs nothing extra in hardware. You do not need a €10k workstation to keep your clients' data private: a normal laptop is enough.
- **Reversible, not destructive.** Classic redaction throws information away. rizzo-pii **pseudonymizes**: the answer you get from the frontier model is reconstructed with the real values, so the tool is useful in real work, not just for compliance theater.

≈0.3B	~0.5 GB	22	0.989
parameters (mmBERT-base)	RAM footprint, CPU	PII categories	micro-F1 (real IT validation)

4 Related work, and how rizzo-pii compares

PII detection is an established task, but the existing open tools are a poor fit for the Italian legal use case.

Microsoft Presidio is the de-facto open framework: a pipeline of spaCy NER plus regular-expression recognizers and checksum validators. It is flexible and extensible, but out of the box it is English-centric and carries no native notion of Italian legal identifiers; getting good Italian-legal coverage means building and maintaining custom recognizers yourself.

OpenAI Privacy Filter (released 2026, Apache-2.0) is the closest analogue in spirit: an on-device, token-classification model for high-throughput data sanitization. It is a strong, modern system: a 1.5B-parameter sparse mixture-of-experts (≈50M active, 128 experts, top-4), 128k-token context, BIOES spans decoded with a constrained Viterbi pass. But it is built for a different audience. It detects **eight** generic categories (`private_person`, `private_address`, `private_email`, `private_phone`, `private_url`, `account_number`, `private_date`, `secret`), it is **primarily English**, and it has **no concept** of the identifiers that dominate Italian legal documents: there is no `codice fiscale`, no `partita IVA`, no `dati catastali`, no protocol/repertory document numbers, no vehicle plate. And despite being "small" in active compute, it still needs **1.5B parameters resident in memory**.

rizzo-pii is deliberately the opposite specialization: **Italian-first** (while still multilingual), **legal-domain** aware, with a richer 22-tag taxonomy, and a **smaller memory footprint**: one dense 0.3B-parameter model (1.2 GB in fp32, 0.5 GB quantized) instead of 1.5B parameters to load. It also pairs the neural model with a deterministic **regex + checksum** safety net (mod-97 for IBAN, Luhn for cards, the official CF/PIVA algorithms) that the larger generic model does not provide.

Property	rizzo-pii:0.3B	OpenAI Privacy Filter	MS Presidio
Type	Dense encoder (mmBERT / ModernBERT)	Sparse MoE encoder	NER + rules pipeline
Parameters	≈0.3B dense (all active)	1.5B total / ≈50M active	spaCy model + rules
Memory to load	0.5–1.2 GB	1.5B params resident	Varies (spaCy)
Runs on	CPU, under 1 GB RAM	On-device	CPU
Categories	**22** (incl. IT-legal)	8 generic	Configurable; EN defaults
Italian CF / PIVA / catasto	**Yes**	No	Not by default
Primary language	Italian (+7 more)	English	English
Checksum validation	Yes (IBAN/CF/PIVA/card)	No	Some recognizers
Reversible mapping	Yes (local dict)	Masking	Anonymization
License / code	Open source	Apache-2.0	MIT

Table 1: rizzo-pii:0.3B versus the most relevant open PII tools. The differentiators are Italian-legal coverage, a smaller memory footprint, and a checksum-backed safety net.

Concrete example

Take the sentence *"Il Sig. Mario Rossi, C.F. RSSMRA85H12F205Z, P.IVA 12345678901, è titolare dell'immobile al Foglio 12, particella 345, sub. 6."* rizzo-pii tags `FULLNAME`, `CF`, `PIVA` and `CATASTO` and rewrites it as *"Il Sig. [FULLNAME_1], C.F. [CF_1], P.IVA [PIVA_1], è titolare dell'immobile al [CATASTO_1]."* A generic English-first model has no label for the fiscal code, the VAT number or the cadastral reference, the three most sensitive identifiers in the sentence, and would leave them in the clear.

5 The taxonomy: 22 tags, and why

rizzo-pii predicts 22 entity types in BIO format (a `B-`/`I-` label per tag, plus `O`). The taxonomy is the product of a deliberate design choice: tag **what a span is** (the thing to mask), not the role it happens to play in a given document. The raw datasets are left untouched; the mapping to these 22 tags is applied at load time through a single `TAG_MAP`, so the taxonomy can be changed in one place without re-annotating anything.

Tag	Meaning	Example	Source
FULLNAME	Person name (incl. legal roles: judge, lawyer, parties, witness)	Mario Rossi	real+synth
AGE	Age	45 anni	real
GENDER	Sex / gender	Femmina	real
DATE	Calendar date	12/06/1985	real+synth
TIME	Time of day	ore 15:30	real
STREET	Street / square	Via Garibaldi	real+synth
BUILDINGNUM	Street number	24	real+synth
ZIPCODE	Postal code (CAP)	00185	real+synth
CITY	City	Milano	real+synth
PROVINCE	Province abbreviation	MI	synth
EMAIL	E-mail (incl. PEC)	m.rossi@studio.it	real+synth
TELEPHONENUM	Phone number	+39 333 1234567	real+synth
CF	*Codice fiscale* (personal tax code)	RSSMRA85H12F205Z	synth
PIVA	*Partita IVA* (VAT number)	12345678901	real+synth
ID_DOC	ID/passport/licence/social number	CA12345AB	real+synth
IBAN	IBAN / bank account	IT60X05428…	synth
CREDITCARDNUMBER	Credit-card number	4111 1111 1111 1111	real
AMOUNT	Money amount	€ 12.500,00	synth
TARGA	Vehicle plate	AB 123 CD	synth
ORG	Private company / firm / bank	Edilnord S.r.l.	synth
DOCID	Act identifier (RG, protocol, repertory, ruling)	1234/2024	synth
CATASTO	Cadastral data (sheet, parcel, sub.)	Foglio 12, part. 345	synth

Table 2: The 22 categories. Five are Italian-legal identifiers (CF, PIVA, CATASTO, DOCID, PROVINCE) that exist in no real public dataset and are supplied entirely by synthesis.

Two design decisions stand out. First, **legal roles collapse into FULLNAME.** Whether "Mario Rossi" is the judge, the lawyer or a witness is not a property of the string: the same name can be a judge in one act and a lawyer in another. Forcing the model to output the role would teach it keyword-matching ("the name after *avvocato*") rather than understanding; the role, if needed, is recovered downstream as metadata. Second, **we merge raw types that mean the same thing**: given names + surnames → FULLNAME; PEC → EMAIL; TAXNUM → PIVA; ID card / passport / licence / social number → ID_DOC; account number → IBAN. Two raw types are dropped to O because they are **not** identifiers to mask: honorifics (Dott., Avv.) and the name of the **court** itself (a public body, kept as context).

The five Italian-legal tags are the reason rizzo-pii exists: CF, PIVA, CATASTO, DOCID and PROVINCE simply do not appear as labeled data in any public corpus, so they are created through synthesis with mathematically valid checksums (see §6).

6 Dataset

The model is trained on a **multilingual** pool of **≈745k** labeled rows assembled from four sources, all remapped to the 22 tags at load time. The guiding principle for the synthetic part is what we call **"LLM-author / code-labeler"**.

> **Key idea: LLM writes the prose, code injects the data**

An LLM (Gemini) writes only Italian legal **prose with placeholders** (`{SLOT}`); our code then **injects** the actual values (fiscal codes, VAT numbers and IBANs computed with their **real checksum algorithms**) into those slots. Because we know exactly where each value was inserted, the BIO labels are exact and free; the checksums are valid by construction; and **no real personal data is ever produced by the LLM.** Three hard problems (label noise, invalid identifiers, PII leakage) solved at once.

6.1 The four sources

Source	Rows	Share	What it contributes
Ai4Privacy `open-pii-masking-500k`	464,124	62.3%	Real human text with masked PII, 8 languages, the multilingual backbone (CC-BY-4.0)
Synthetic from templates (ours)	200,000	26.8%	Italian legal prose + injected, checksum-valid values; covers the IT-legal tags
Augment in real text (ours)	40,000	5.4%	Synthetic entities injected into **real** Ai4Privacy Italian sentences, at variable positions
DeepMount `pii-masking-ita`	40,788	5.5%	Faker data translated to Italian; varied non-legal context for IBAN/ORG/AMOUNT/TARGA

In total, **240,000 rows (32.2%) are generated by us**; counting DeepMount's Faker material, **37.7%** of the pool is synthetic and the remaining **62.3%** is the real, multilingual Ai4Privacy reference set. The synthetic data is what makes the Italian-legal tags learnable at all; the real and augmented data is what keeps them grounded in natural prose and mitigates structural over-fitting.

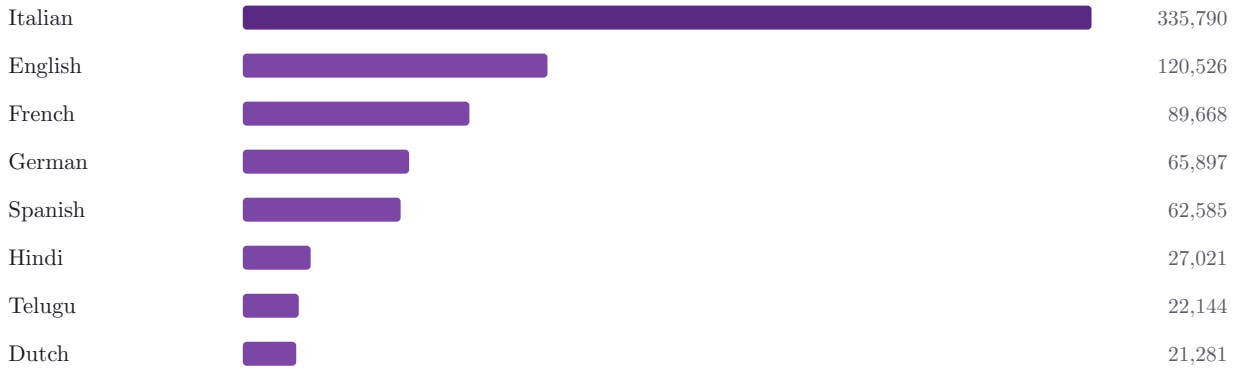
6.2 Size of the corpus

744,912	~67.8 M	~105.9 M	~5.70 M
training rows	words	mmBERT subword tokens	tagged entities

The corpus is 334 M characters; the average row is 142 subword tokens, the token/word expansion is 1.56×, and the longest single sequence is **962 tokens** (a multilingual Ai4Privacy example). This is what motivates the `MAX_LEN = 768` setting: it admits essentially all of the long synthetic acts (only 1% of the synthetic set is truncated) while staying far inside mmBERT's native 8192-token window. A held-out **validation set of 7,000 real Italian rows** is used for all metrics (see §8).

6.3 Language distribution

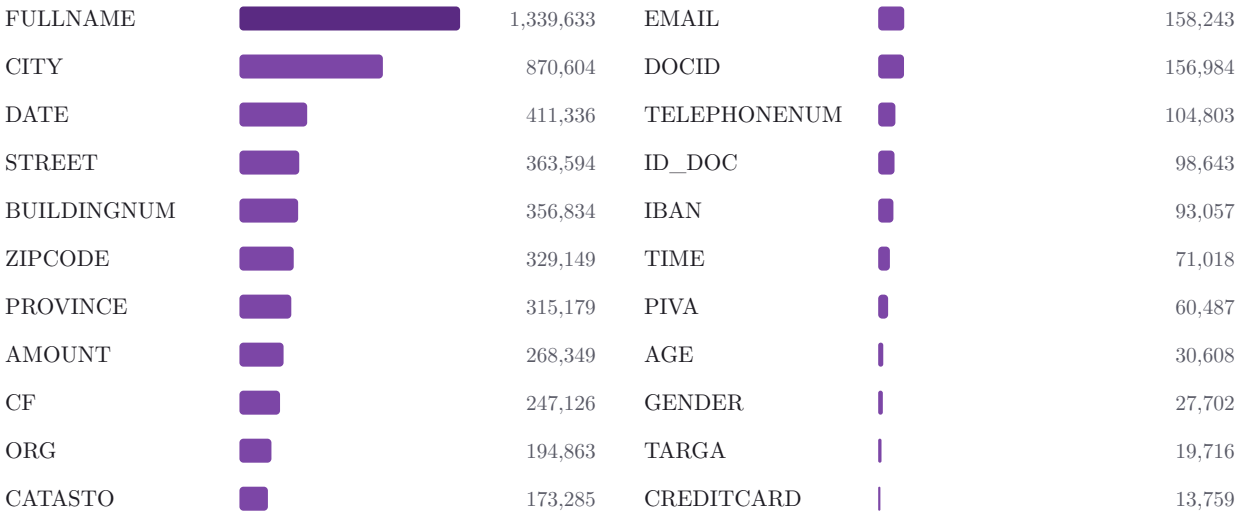
Because every synthetic and DeepMount row is Italian, Italian becomes the most represented language of the pool (45%), even though training stays genuinely multilingual across 8 languages.



Bars are scaled to the largest language (Italian). Italian = 45.1% of the pool; non-Italian = 54.9%.

6.4 Tag distribution

The 5.70 M tagged entities are dominated by names and locations; the rare end (`CREDITCARDNUMBER`, `TARGA`) is 97× smaller than `FULLNAME`, which is worth remembering when reading per-tag metrics.



7 Model and training setup

The backbone is **mmBERT-base** (`jhu-clsp/mmBERT-base`), a multilingual encoder that follows the **ModernBERT** architecture: 22 transformer layers, hidden size 768, 12 attention heads, alternating local/global attention, rotary embeddings, and a **native 8192-token context**. We chose mmBERT over vanilla ModernBERT because the latter is almost exclusively English, whereas our use case is Italian-and-multilingual. The 256k-token multilingual vocabulary is the main reason the model lands at ≈0.3B parameters. A token-classification head over 44 BIO classes is fine-tuned on top.

Setting	Value	Setting	Value
Backbone	mmBERT-base (ModernBERT)	Optimizer	AdamW
Parameters	≈0.3B	Learning rate	5e-5
Task	Token classification (BIO)	Warmup ratio	0.05
Output classes	44 (BIO)	Weight decay	0.01
Max length	768 subwords	Epochs	1
Precision	bf16	Effective batch	28 (14 × 2 accum.)
Context (native)	8192	Length grouping	yes (precomputed)

Table 3: Training configuration of the reported checkpoint.

Hardware and time. Training was done on a single consumer GPU, an **NVIDIA RTX 5060 Ti, 16 GB** (Blackwell, sm_120), on Windows, with PyTorch built for CUDA 12.8 (the Blackwell architecture requires a `cu128` build; cpu/cu121 wheels do not support sm_120). One full epoch over the pool took **under two hours**. VRAM is the binding constraint on a 16 GB card shared with the desktop: at `MAX_LEN=768` a dense batch can saturate memory and send the CUDA allocator into thrashing, so the recipe uses a modest per-step batch with gradient accumulation (effective 28), `expandable_segments` allocation, and length-grouped batching with **precomputed** lengths to avoid re-tokenizing the lazy dataset. Evaluation of the 7,000-row validation set takes 24 s.

8 Results

8.1 Training dynamics

Training was a single epoch (26.6k optimizer steps over 744,912 rows). The loss fell fast and cleanly, kept improving to the very end, and showed no sign of over-fitting.

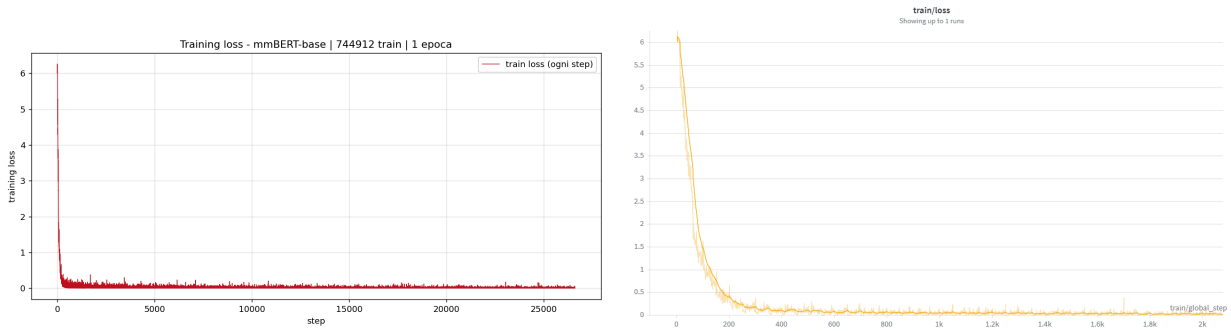


Figure 2: Training loss over the full epoch (26.6k steps). **Left:** every step, raw; **right:** a smoothed zoom. It drops from ≈6.2 to below 0.1 within the first few hundred steps, then settles into a clean, monotone, low and stable regime with no divergence or oscillation.

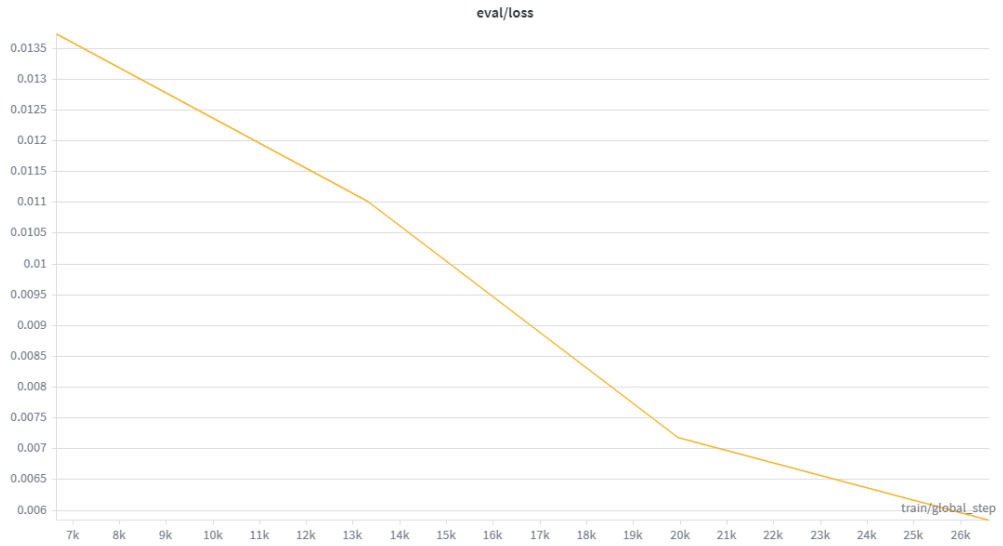


Figure 3: Validation loss, evaluated four times across the epoch: it decreases monotonically (≈0.0137 down to 0.0058) and is **still falling when training stops**.

The numbers confirm the picture: a final training loss of **≈0.003** and a validation loss of **≈0.006** are both very low and very close, so the model is **not over-fitting**. The tiny train/val gap is exactly what a healthy fit looks like, and the validation curve is monotone, never turning back up. Crucially, the validation loss was **still decreasing** when the single planned epoch ended, not because the model had converged: **more steps, or a second epoch, would very likely have pushed it lower still**, a cheap and obvious lever for the next run.

8.2 Per-tag accuracy on real Italian validation

On the 7,000-row held-out Italian benchmark the model reaches **0.989 micro-F1**, with **0.987 precision** and **0.990 recall**, and a token accuracy of **0.998**. The unweighted per-tag mean (macro-F1) across all 22 tags is **0.987**, and every one of the five Italian-legal identifiers scores a perfect 1.000.

0.987	0.990	0.989	0.998
micro precision	micro recall	micro F1	token accuracy

Tag	Sup.	P	R	F1	Tag	Sup.	P	R	F1
FULLNAME	4390	.989	.990	.990	GENDER	472	1.00	1.00	1.00
CATASTO	1200	1.00	1.00	1.00	PROVINCE	400	1.00	1.00	1.00
CITY	953	.961	.963	.962	DOCID	400	1.00	1.00	1.00
DATE	922	1.00	1.00	1.00	CF	400	1.00	1.00	1.00
TELEPHO-NENUM	874	1.00	1.00	1.00	AGE	385	.979	.977	.978
ID_DOC	800	1.00	1.00	1.00	ZIPCODE	299	.938	.967	.952
EMAIL	748	.999	.999	.999	IBAN	278	.996	.996	.996
TIME	637	.991	.992	.991	CREDIT-CARD	257	.919	.973	.945
STREET	617	.951	.969	.960	AMOUNT	146	1.00	.993	.997
BUILD-INGNUM	594	.969	.958	.964	ORG	145	.967	1.00	.983
PIVA	514	1.00	1.00	1.00	TARGA	43	1.00	1.00	1.00

Table 4: Per-tag precision / recall / F1 for all 22 tags on `validation_real.jsonl` (7,000 real held-out Italian rows; 15,474 entities), model v1.2.0.

All five Italian-legal identifiers (`CF`, `PIVA`, `CATASTO`, `DOCID`, `PROVINCE`) score a perfect **1.000**, as do `ID_DOC`, `DATE`, `TELEPHONENUM`, `GENDER` and `TARGA`: the categories that no other open model even covers are the ones rizzo-pii nails. The remaining soft spots are the open, high-variability classes (`ZIPCODE`, `CREDITCARDNUMBER`, `STREET`, `CITY`) and `ORG`, which is exactly where a larger, better-balanced dataset would help (see §10–§11).

9 Deployment: it runs on a normal computer

The released checkpoint is 1.2 GB on disk in fp32 and runs comfortably on a **CPU**: quantized, its memory footprint is on the order of **0.5 GB**, with **no GPU required**. That is the whole point: the privacy layer is cheap enough to run on the same laptop the user already owns. Packaging is provided as a desktop application: a native window (**Rizzo PII**, Tauri/WebView2) that launches the Python/Flask backend as a bundled CPU "sidecar", plus a per-user Windows installer; a CPU-only PyTorch build keeps it fully **offline**.

In production the neural model is never used alone. It is paired with a deterministic **regex + checksum** network for the structured identifiers (EMAIL, phone, IBAN, CF, PIVA, credit card, amount, plate), where IBAN/PIVA/card must pass their **checksum** (mod-97 / Luhn) to be accepted, and a valid checksum **overrides** the model. This eliminates the classic failure mode of a neural tagger fragmenting a long code, and gives mathematically certain detection for exactly the identifiers whose leakage is most damaging. The app itself adds the reversible layer (stable placeholders, downloadable local dictionary, a "restore" tab tolerant to markdown/format drift), chunking with overlap for long PDFs, and a colored per-tag UI.

9.1 Minimum requirements

To use the model (inference)

- Any 64-bit CPU (no GPU)
- 0.5–1.2 GB RAM for the model
- Windows (installer) / Linux / macOS
- Fully offline; no API key

To retrain the model

- A single 16 GB GPU is enough
- Reference run: RTX 5060 Ti, 2 h
- PyTorch `cu128` for Blackwell
- 745k rows, regenerable from scripts

10 Limitations

We state the boundaries plainly:

- **Validation is Italian-only.** Training is multilingual, but the 7,000-row benchmark measures Italian only, by design, since the target domain is Italian legal text. The other seven languages are trained but not certified.

- **The IT-legal tags are validated against injected entities.** `CF`, `PIVA`, `CATASTO`, `DOCID`, `PROVINCE` have no real public data, so even in validation they are generated entities placed into real sentences: a good proxy, not a fully blind test.
- **Class imbalance and under-represented categories.** The corpus is heavily skewed (`FULLNAME` outnumbers `CREDITCARDNUMBER` 97×), so the rarer tags are noisier and should lean on the checksum net in production. The clearest case is **organizations** (`ORG`): company, firm and bank names are an open, highly variable class, yet today they come largely from synthetic templates and off-domain Faker data, with little real Italian coverage, so this is where the model is most likely to slip, and where a larger, **balanced** and **representative** dataset would help most (see §11).
- **Off-domain synthetic values.** DeepMount supplies US-style names/addresses, useful for form and context, not as Italian values.
- **Evaluation is sentence-level, not document-level.** The 7,000-row benchmark is built from short sentences, whereas the real use case is **whole** documents: multi-page acts, contracts and rulings. Measuring true end-to-end behaviour (long context, PDF chunking with overlap, real act structure) needs a dedicated test set of **large, real Italian documents**, which still has to be assembled.

The mitigation that matters in practice: **always pair the model with the regex/checksum safety net.** The two together are stronger than either alone.

11 Next step: a community-owned Italian PII dataset

rizzo-pii proves the thesis: you **can** keep using frontier models and still keep your data private, on ordinary hardware, with Italian-legal coverage no other open model offers. The clearest path to making it better is **data**. The single biggest lever on quality is a large, varied, **real** (and lawfully collected) Italian corpus: both for the legal identifiers that are scarce today (the part we have had to synthesize) and, above all, to **balance the classes** and add genuine coverage where the model is weakest: **organizations** (company, firm and bank names), which are far too varied to learn from templates alone. We also need a **test set of large, real documents** (whole acts, contracts and rulings, lawfully shared), so the model can be measured end-to-end on the documents it is actually meant to anonymize, not only on isolated sentences.

So the project is **open source**, and this is an open invitation. If you work with Italian documents (lawyers, accountants, notaries, developers, researchers), help build a **shared Italian PII dataset**: contribute templates, annotation, edge cases, and review. A privacy tool for Italy is something Italy should build together, for the good of everyone's privacy.

12 Dataset contributors

The community dataset `rizzoaiacademy/anonimizzazione-testi-italiano` is built by people who ran the generator and opened a pull request on Hugging Face. The contributors below have had contributions **merged** into the dataset.

Francesco Wfm @workingfm	**Craicek** @Craicekhf	**Lucio Palmieri** @plucius
Lorenzo Ferrari @Lorenzfer	**Yuri** @D3ros	**Daniele Murabito** @worksdem
Fabio Accalai @fabos	**Igino** @ijinx	**Attilio Pregnolato** @8Attilio1
Andrea @Renaad	**Matteo Scortegagna** @Mascorte0	**Marco Sinatra** @CapitanJackMarcoS
Carlos H. P. Ross @WolCarlos	**Alessandro Betti** @bettialessandro	**Marco Occhialini** @occhials

Maintainer commits (Simone Rizzo) are excluded.

Contributions under review. A further 15 people have open pull requests that are not merged yet, and are gratefully acknowledged here as well: Lino (@Lino9999), Raffaele Francesco D'Amato (@kekkodamato), Francesco (@francechian), Luca (@LucaVes), ebbasta (@esseggi), Emiliano (@Darkfog81), Luca Mazzola (@LuCaMazZ), Pietro Bardone (@p3pp01), Marco Buzzanca (@Eagle-lab), Di Gesto (@Filippo76), Gianluca Vertemati (@GiaVer), Massimiliano (@MassiLLM), go hatfd (@blokka11), flyer (@maxxflyer), nicola (@najmarte).

Contribute to the project

github.com/Rizzo-AI-Academy/rizzo-pii

Star it, open an issue, send a pull request, or just share a hard example.

Author. Simone Rizzo. **Sponsor.** Rizzo AI Academy (rizzoaiacademy.com).
The mascot, a hedgehog, guards the document and stays inside the EU. Model name: **rizzo-pii:0.3B**. Built and trained in Italy.

References

[1] mmBERT: a multilingual ModernBERT encoder. JHU-CLSP. `jhu-clsp/mmBERT-base`, Hugging Face.
[2] Warner, B. et al. *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder* (ModernBERT), 2024.
[3] Ai4Privacy. *open-pii-masking-500k*. Hugging Face (CC-BY-4.0).
[4] DeepMount00. *pii-masking-ita*. Hugging Face.
[5] OpenAI. *Introducing OpenAI Privacy Filter*, 2026; model `openai/privacy-filter`, Hugging Face (Apache-2.0).
[6] Microsoft. *Presidio: Data Protection and De-identification SDK*. Open source (MIT).
[7] Regulation (EU) 2016/679: General Data Protection Regulation (GDPR).
[8] Regulation (EU) 2024/1689: Artificial Intelligence Act (EU AI Act).